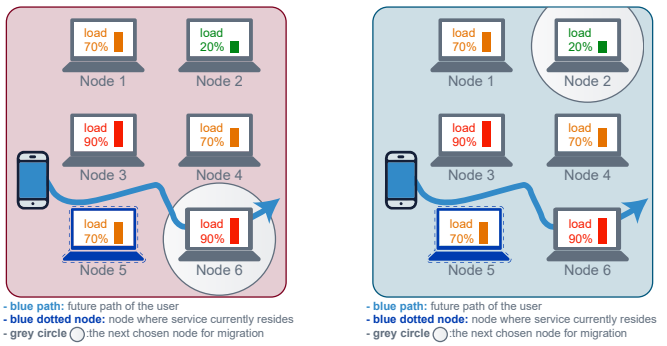
Proactive Migration for Dynamic Computation Load in Edge Computing

Amr M. Zaki*, Sameh Sorour*
* School of Computing, Queen's University, Kingston, ON, Canada
Email: {amr.zaki, sameh.sorour}@queensu.ca

ICC 2022 - IEEE International Conference on Communications | 978-1-5386-8347-7/22/$31.00 ©2022 IEEE | DOI: 10.1109/ICC45855.2022.9838613

***Abstract*—The advent of the Internet-of-Things (IoT), which streams a wide range of computation-intensive applications with strict Quality of Service (QoS) requirements, has caused a paradigm shift from cloud computing to edge computing. Edge computing can drastically reduce latency and improve QoS. However, various dynamic changes can affect service continuity, thus requiring service migration. The dynamic computation load is one of the changes that are typically overlooked in service migration. In this paper, we propose the Dynamic Load-based Proactive Migration (DLPM) scheme. DLPM adopts a finite-state machine (FSM) that models the dynamic computation load, and proactively migrates computation tasks based on the associated transition probabilities. We formulate the service migration problem as an integer linear programming (ILP) optimization problem that aims to minimize the delay. We provide an analytical solution to the optimization problem using the KKT conditions and Lagrangian analysis. Performance evaluation shows that DLPM yields significant improvements in terms of delay and number of migrations compared to the reactive migration approach.**

***Index Terms*—Proactive Migration, Fog Computing, Edge Computing.**



(a) Service is migrated to the nearest node, without accounting for the dynamic computation load (node6)

(b) (DLPM) Service is migrated to the nearest node, with the lowest computation load (node2)

Fig. 1: Service Migration using different migration schemes

I. Introduction

With the proliferation of the IoT technology that enables the vision of smart cities, smart grids, smart homes, etc., it is expected that 41 billion IoT devices will be connected to the Internet by 2027 [1]. In addition, the IoT market size is expected to amplify in the future, with investments reaching up to $1.1 trillion by 2023 [1]. This expansion is expected to foster a broad range of applications, including time-sensitive applications such as healthcare, that require strict QoS requirements (high availability, low latency, low communication cost). Cloud computing has been used in addressing the needs of such applications. However, such data centers are often geographically distant from the end user, making it prone to high network delay which would hamper the QoS provided to the user [2]. Edge computing has been introduced as a promising solution to mitigate these challenges [3]. Due to their proximity to end users, leveraging the computational resources available at the edge can drastically reduce latency. This can significantly improve the QoS provided to users.

Despite its leverage, edge computing can encounter some dynamic changes that can lead to dramatic decrease in QoS, significant degradation in network performance, and interruption of ongoing edge services. Such dynamic changes are typically attributed to the mobility of users and the limited coverage of edge servers [4]. In order to cope with these changes, migration policies are needed in order to migrate the ongoing service to the nearest node (i.e., edge server) that is capable of handling the user's request. Such policies need to balance between performing frequent migrations and low number of migrations. This is since the former can cause service downtime, while the latter can increase latency, since it can cause the service to continue to be executed too far from the user [3]. Thus, these migration policies need to carefully determine when to migrate and to which edge node.

Different migration elements have been studied throughout literature, ranging from tasks to services to virtual machines (VM) [3]. However, throughout this work, we refer to the VM begin migrated as a service, as it this is most used term in literature. Existing service migration policies can be categorized based on when to migrate into reactive and proactive policies. In the former, migration occurs after a need manifests [4]. In the latter, the need for migration is anticipated in advance, and migration decisions are made beforehand [5], [6]. In Reactive approaches the migration starts when the handoff process is concluded. This can in fact cause prolonged delays, as after the handoff is concluded, the user would still be accessing the source edge node using the new access point, hence more than one hop is needed to access his service [7]. However, in proactive approaches, the migration starts before the handoff. This enables the service to be available at the destination edge node once the handoff is concluded [7].

Most of the existing proactive migration schemes consider

the dynamic changes that trigger service migration from the perspective of users' mobility. In particular, they focus on mobility prediction to address the service migration issue [8], [6]. This helps migrate the service to the user's future position in advance.

In contrast to the users mobility factor, changes caused by the dynamic computation load at edge nodes have been mostly overlooked in existing service migration schemes. Note that the computation capability of an edge node is significantly reduced if it is experiencing a heavy computation load. Thus, its ability to serve a newly migrated service can be heavily compromised. Since such computation load is highly dynamic, it is crucial for service migration to take this dynamic nature into consideration. In this paper, we take the dynamic computation load into consideration by proposing the Dynamic Load-based Proactive Migration (DLPM) scheme. In DLPM, we rely on probabilistic estimated changes in the computation load of edge nodes to proactively find the optimal nearest edge node that has the highest probability of having the lowest load. This is to ensure selecting the edge node that has the highest computation capability to serve the migrated service. As depicted in Fig. 1a, most of the existing service migration schemes, only consider migrating to the nearest edge node, without accounting for the dynamic computation load. However, as shown in Fig. 1b, DLPM accounts for both the distance to the edge nodes, and their computation load when migrating the service. Hence choosing the nearest most computationally capable edge node. DLPM scheme was formulated as an integer linear program (ILP) optimization problem, an analytical solution using Karush–Kuhn–Tucker (KKT) and Lagrange analysis has been provided.

Extensive simulation experiments were conducted using MobFogSim [7] to assess the performance of our proposed scheme (DLPM). We compared DLPM to a baseline reactive approach (lowest latency approach). The results have shown that DLPM significantly outperforms the reactive approach in terms of a lower average delay across different number of users while also maintaining a lower number of average migrations.

This article is organized as follows: Section II highlights some of the related work in service migration. Section III-A provides a detailed description of the proposed service migration scheme (DLPM). Section IV presents the performance evaluation and simulation results. Section V illustrates our conclusions and future work.

II. Related Work

Different migration strategies have been proposed in the literature. They can be categorized into reactive [4] and proactive migration schemes [6], [5]. Reactive schemes make migration decisions in response to the occurrence of a triggering event [4]. In contrast, proactive schemes strive to anticipate and identify the need to migrate in advance [6]. Choosing one strategy over the other, mainly depends on the service and its considerations. In [2], it has been shown that proactive migration can significantly reduce the delay compared to the reactive approach when the migrated data is large. In [9], it has been demonstrated that proactive migration is better in content dissemination compared to the reactive approach. Other research efforts considered a hybrid approach [4], where the reactive approach is used in case of unexpected events that are hard to predict due to lack of any repetitive patterns, whereas the proactive approach is adopted when triggering events are repetitive and can be predicted.

In proactive migration, a migration decision is built on predicting the best node to migrate the service to. Most of the existing proactive migration schemes focus on proactively predicting the user's mobility. This stems from the fact that many users tend to have a daily routine. Thus, they can have highly predictable movement patterns, particularly when using public transportation, such as buses, trains, etc. Many mobility prediction schemes have been proposed [10] [6] [8] [11]. Some of these schemes adopt a Markovian-based approach for prediction, such as Markov chains [10], and n-mobility Markov chain (n-MMC) [6], whereas others use deep learning approaches, such as Long-Short Term Memory (LSTM) [8], and by combining the Long-Short Term Memory networks (LSTM) with Reinforcement Learning (RL) in (RL-LSTM) [11]. Mobility predictions, along with other attributes (signal strength, quality of Service) [11] are then used to make the migration decision in order to choose the best node to migrate the task to.

Many of the existing schemes model the migration decision as an optimization problem [5], [12], [13], [14], while others use a heuristic-based approach [6] [15].

Despite the demonstrated leverage of proactive migration schemes, investigating the dynamic computation load as a triggering event has been mostly overlooked in such schemes. In this paper, we consider the dynamic load by proposing a proactive migration scheme (DLPM) that incorporates the use of a finite state machine (FSM) to model the transition from one possible load state to another at each edge node in order to make optimal proactive migration decisions.

III. Dynamic Load-based Proactive Migration (DLPM)

In this section, we provide a detailed description of the system model, as well as the ILP optimization problem.

A. System Model

Given a set of edge nodes N= $\{n_1, n_2, \dots, N\}$ and a set of services for the mobile devices D= $\{d_1, d_2, ..., D\}$ at any given time. Each service has a certain computation workload or intensity measured in number of instructions executed (batch size), denoted β. We model the dynamic load at each edge node using a state machine. As depicted in Fig. 2, each edge node n_i can be in one of 3 possible states, each of which corresponds to a potential computation capability, expressed in terms of million instructions per second (MIPS) ν_{si}. Such states reflect high, medium, and low MIPS. At any time, for each edge node n_i , each state k of its possible states triggers a different MIPS rating ν_{si}.

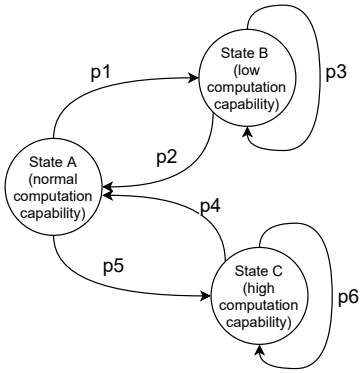


Fig. 2: State machine model of the dynamic load

Our proposed scheme DLPM aims to proactively choose the nearest edge node to the user with the highest probability of having the highest computation capability (i.e, the highest MIPS rating) to best serve the migrated service. Our objective is to maximize the difference between the expected delays of both the end user and the current edge node (i.e., the edge node currently serving the mobile device) and that between the end user and the potential next edge node to which the task can migrate. If the latter's expected delay is lower than that of the former, migration should be triggered. Solving the problem periodically (each $t_r = 1$sec), enables the system to identify when and where to migrate.

The total delay between the end user and any edge node, denoted t_{dn} is given by Eq. 1, where c_{dn} is the computation delay, and z_{dn} is the propagation delay. The computation delay is the time taken by edge node n to compute task of mobile d, whereas the propagation delay is the time taken for the service to reach its destination. Transmission delay is ignored, as data rate is assumed to be the same for all devices across all nodes.

$$t_{dn} = c_{dn} + z_{dn} \tag{1}$$

The transition probabilities ψ_{si} of the aforementioned finite state machine (FSM) Fig. 2, are assumed to be known. These are used to compute the expected computational delay of the service of mobile device d on the edge node n, which is given by Eq. 2.

$$\begin{aligned} c_{dn} = (\beta/\nu_{s1}) \times \psi_{s1} \\ +(\beta/\nu_{s2}) \times \psi_{s2} \end{aligned} \tag{2}$$

The proposed optimization model aims to optimize expected differential delay x_{dn} between the total delay of the mobile device d and the current occupied node (n=1) t_{d1}, and the total delay of the mobile device d with a new node (n=2) t_{d2} Eq. 3.

$$x_{dn} = t_{d1} - t_{d2} \tag{3}$$

B. Problem Formulation

Our objective is to minimize the delay by migrating the service to the nearest edge node that has the lowest computation load (i.e., highest MIPS rating). The optimization problem is formulated as a 0-1 Integer Linear Program (0-1 ILP). The decision variable is in the form of a placement matrix, denoted P_{dn}, where p_{ij} if service i is placed on edge node j, and 0 otherwise. The optimization problem is given by Eq. 4.

$$\max_{p} \quad \sum_{d=0}^{D} \sum_{n=0}^{N} p_{dn} x_{dn} \tag{4a}$$

$$\text{s.t.} \quad \sum_{d=0}^{D} p_{dn} \leq 1 \qquad \forall n \in N \tag{4b}$$

$$\sum_{n=0}^{N} p_{dn} = 1 \qquad \forall d \in D \tag{4c}$$

$$\sum_{d=0}^{D} p_{dn} m_d \leq M_n \quad \forall n \in N \tag{4d}$$

$$\sum_{\substack{d=0 \\ x_{dn} \neq 0}}^{D} p_{dn} e_d \leq E_n \quad \forall n \in N \tag{4e}$$

$$p_{dn} x_{dn} \leq \tau \quad x_{dn} \neq 0 \quad \forall d \in D \quad \forall n \in N \tag{4f}$$

$$p_{dn} = 0 \quad \text{or} \quad 1 \qquad \forall d \in D \quad \forall n \in N \tag{4g}$$

Constraint (4b) ensures that each edge node can have at most one migrated service. Constraint (4c) specifies that each service should be allocated to only one edge node. This edge node is either the new node to which the task migrates or the current node at which the task resides (if no migration occurs).

Constraints (4e) and (4d) are used to ensure that the memory requirements (m_d) and the energy consumption (e_d) of the migrated service do not exceed certain predefined thresholds, denoted M_n and E_N, respectively. Note that if no migration occurs, which is the case when $X_{dn} = 0$ the current node will be selected regardless of the energy constraint.

A special case occurs when two edge nodes that are close to each other have similar MIPS rating (ν_{si}) (i.e., similar computation load). This causes an oscillating effect, where the model keeps on oscillating between the two nodes. To avoid such a behavior, constraint (4f) is added to prevent migration to a new node when the value of $p_{dn}x_{dn}$ is smaller than a predefined threshold (τ). This constraint only applies when $x_{dn} \neq 0$ (i.e., a migration is needed) since if no migration occurs, the current edge node is selected regardless of the threshold.

Constraint (4g) ensures that each element p_{dn} in the binary placement matrix P is either 0 or 1.

C. Analytical Solution

The proposed optimization problem is a 0-1 Integer Linear Program (0-1 ILP). It is a modification to the generalized assignment problem (GAP), which is well known to be NP-hard [16]. Thus, we consider one of the relaxations proposed by [16], which is the linear program relaxation (LP-Relaxation). It is obtained by replacing constraint (4g) with $0 \leq p_{dn} \leq 1$, which converts the problem to an ILP.

The Lagrangian function associated with the aforementioned optimization problem in Eq. 4 is given by the following expression Eq. 5

$$L(P,\lambda_n,\mu_n^{(1)},\mu_n^{(2)},\theta_{dn},\omega_{dn}^{(1)},\omega_{dn}^{(2)}) = -\sum_{d=0}^{D}\sum_{n=0}^{N} p_{dn}x_{dn} + \sum_{n=0}^{N}\lambda_n^{(1)}\left((\sum_{d=0}^{D} p_{dn})-1\right) + \sum_{d=0}^{D}\lambda_d^{(2)}\left((\sum_{n=0}^{N} p_{dn})-1\right) + \sum_{n=0}^{N}\mu_n^{(1)}\left((\sum_{\substack{d=0\\ x_{dn}\neq 0}}^{D} p_{dn}e_d)-E_n\right) + \sum_{n=0}^{N}\mu_n^{(2)}\left((\sum_{d=0}^{D} p_{dn}m_d)-M_n\right) + \sum_{n=0}^{N}\sum_{\substack{d=0\\ x_{dn}\neq 0}}^{D}\theta_{dn}(p_{dn}x_{dn}-\tau) + \sum_{n=0}^{N}\sum_{d=0}^{D}(\omega_{dn}^{(1)}(p_{dn}-1)) - \sum_{n=0}^{N}\sum_{d=0}^{D}(\omega_{dn}^{(2)}(p_{dn})) \tag{5}$$

Where $\lambda_n^{(1)*}, \lambda_d^{(2)}$ are the associated Lagrange multipliers to the placement constraints (4b) and (4c). $\mu_n^{(1)}, \mu_n^{(2)}$ are the associated Lagrange multipliers to the energy and memory constraints (4e) and (4d). θ_{dn} is the associated Lagrange multiplier matrix for the oscillating threshold τ (4f). $\omega_{dn}^{(1)}, \omega_{dn}^{(2)}$ are the associated Lagrange multipliers for the relaxed binary placement constraint (4g).

Theorem 1: Using Karush-Kuhn-Tucker (KKT) conditions, an analytical solution of the problem is found.

$$\begin{cases} p_{dn}^* = \tau/x_{dn} & x_{dn}(1-\theta_{dn}^*) = \lambda_n^{(1)*} + \lambda_d^{(2)*} + \mu_n^{(1)*}e_d + \mu_n^{(2)*}m_d \\ p_{dn}^* = 0 & x_{dn}(1-\theta_{dn}^*) < \lambda_n^{(1)*} + \lambda_d^{(2)*} + \mu_n^{(1)*}e_d + \mu_n^{(2)*}m_d \\ p_{dn}^* = 1 & x_{dn}(1-\theta_{dn}^*) > \lambda_n^{(1)*} + \lambda_d^{(2)*} + \mu_n^{(1)*}e_d + \mu_n^{(2)*}m_d \end{cases} \tag{6}$$

Proof. The proof of Theorem 1 is in (Appendix A) ■

This solution depends on the Lagrangian multipliers, so no closed answer can be directly found. In order to solve this optimization problem, the optimization solver IBM CPLEX [17] is used.

IV. Performance Evaluation

In this section, we evaluate the performance of DLPM compared to the Lowest Latency (LL) scheme [7], which is a baseline reactive migration scheme that triggers migration whenever the user moves away from the current edge node. We use the following performance metrics: 1) the average delay experienced starting from the time a request is sent until a response is received, and 2) the average number of migrations triggered by the system. As the number of migrations increases, the system can experience more down time because of the migration overhead, so it is better to have lower number of migrations in the system.

TABLE I: List of Simulation Parameters

Parameters	Value
Simulation Time (t_s)	10 mins
Frequency of MIPS State Change (t_f)	30 secs
Frequency of Running the Optimization Solver (t_r)	1 sec
Number of Edge Nodes	144
Batch size (β)	20k million instructions (different batch sizes will be mentioned later)
Number of Users	20 users (different number of users will be mentioned later)
Oscillating Threshold (τ)	3 and 5
State A (normal MIPS)	3234 MIPS
State B (low MIPS)	646 MIPS
State C (high MIPS)	4851 MIPS
Transition probabilities	p1=0.7, p2=0.3, p3=0.7, p4=0.1, p5=0.3, p6=0.9

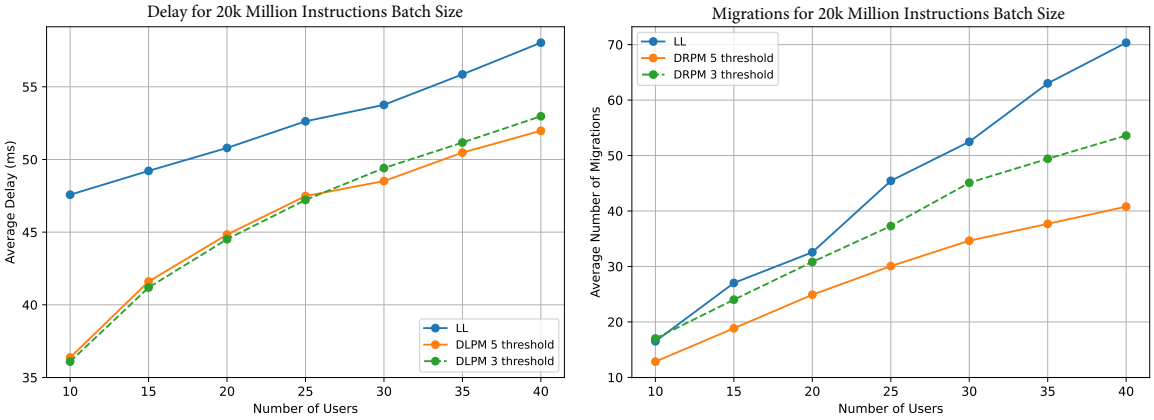
A. Simulation Setup

DLPM and LL have been implemented using the MobFogSim simulator [7], which is a prominent Java-based simulator that is capable of simulating service migration between mobile users and edge nodes. MobFogSim is integrated with the IBM CPLEX optimization solver [17] to generate the optimal solution in DLPM, where the model runs every 1 second (i.e., t_r =1 sec). Realistic mobility traces are used by exploiting the Luxembourg SUMO traffic dataset (LuST) [18], which contains the mobility patterns of buses with average speed of 22.3 kmph in routes of 26.44 min on average. A map of 16km x 16km is considered, which consists of 144 edge nodes. Unless otherwise specified, the number of users is set to 20 and the batch size is set to 20k Million instructions. The simulation period is set to 10 minutes (i.e., t_s = 600 seconds). The computation load of each edge node changes every 30 seconds (i.e., $t_f = 30$ seconds) according to the finite state machine. Experiments are conducted using two different oscillating thresholds (τ) to study its effect on controlling the number of migrations. In particular, τ is set to 3 and 5. Table I summarizes the simulation parameters.

B. Simulation Results and Analysis

In our experiments, we evaluate the performance of DLPM over varying number of users, as well as varying number of batch size. Simulation results are presented at a confidence level = 95% . The acquired results are presented below.

1) Results Over Varying Number of Users: We compare DLPM and the reactive LL scheme in terms of average delay over varying number of users. This is done for different values of τ (3 and 5). As shown in Fig. 4, as the number of users increase, the system becomes more congested, hence the average delay increases. Also as depicted in Fig. 3a, DLPM yields significant reduction in the average delay compared to LL. This can be attributed to the fact that the proposed optimization model strives to migrate the services to the nearest nodes that have low computation load. This leads to selecting selecting more computationally capable edge nodes, which results in faster processing of the migrated service, hence the lower average delay. In contrast, LL only migrates



(a) Delay over varying number of users

(b) Migrations over varying number of users

Fig. 3: Delay and Migrations for service of size 20k million instructions over varying number of users

to the nearest nodes, without accounting for the dynamic computation load, hence the higher average delay. It is also evident that the τ has a negligible effect on the average delay, except for a slightly noticeable impact as the number of users increases. This occurs because when the oscillating threshold increases, migration becomes more restrictive and it tends to favor the selection of better nodes (closer, more computationally capable), which leads to lower delay.

We conduct the same comparison in terms of the number of migrations Fig. 3b. The average number of migrations increases as the number of users increases (in both DLPM and LL). The average migrations in LL is caused by the mobility of users, which changes at a faster rate than the dynamic load, thus increasing the rate at which migration is required. However, DLPM accounts for both mobility and the dynamic load and is able to migrate the services to the nearest most computationally capable edge nodes, hence the lower number of average migrations. The number of migrations is primarily controlled by the oscillating threshold (τ) as it increases, the migration process becomes more restrictive, thus reducing the number of migrations.

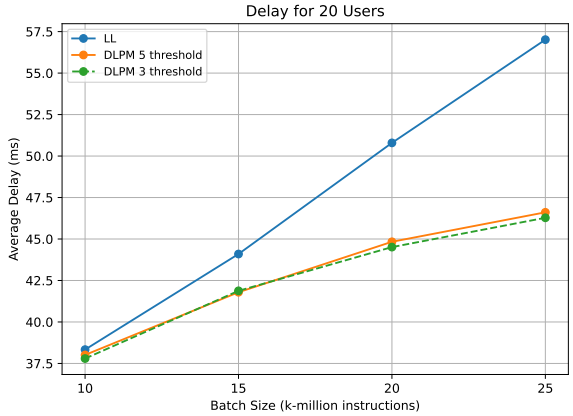


Fig. 4: Delay for 20 users over varying batch size services

2) Results Over Varying Batch Sizes (β): In this experiment, we evaluate the performance of DLPM in terms of the average delay compared to LL under varying batch sizes (β). As depicted in Fig. 4, DLPM exhibits significant reduction in delay compared to LL. This is due to the same aforementioned reasons. Also as shown in Fig. 4, as the batch size increases, the delay increase for both DLPM and LL. This can be attributed to the fact that when the service size increases, it needs more processing time, hence the increase delay. The gap between DLPM and LL increases even further as the batch size increases, with DLPM yielding much lower delay. This is because as the service requirements increase, the need for migrating to more computationally capable nodes become more pressing, which is a factor that DLPM, as opposed to LL, takes into consideration. It can also be noted that varying τ in DLPM has a negligible impact on the delay even as the batch size increases. This is due to that τ is only used to control the number of migrations in the system.

V. Conclusion and Future Work

In this paper, we have proposed the Dynamic load-based Proactive Migration (DLPM) scheme, which considers a typically overlooked factor in migration decisions, which is the dynamic computation load. DLPM proactively migrates services to the nearest edge nodes with the highest probability of having low computation dynamic load (i.e., high computation capability). We have formulated the migration problem as an ILP optimization problem and provided an analytical solution using the KKT and Lagrangian analysis. Performance evaluation has shown that DLPM achieves significant improvements in terms of average delay while maintaining lower number of migrations compared to the baseline reactive LL scheme. The results have also shown that DLPM reduces the delay even further compared to LL when dealing with service with high computation requirements (i.e., large batch size β).

In our future work, we plan to account for both the future users' position (i.e., dynamic mobility of users) and the dynamic computation load.

ACKNOWLEDGMENT

This research is supported by a grant from the Natural Sciences and Engineering Research Council of Canada (NSERC) under grant number ALLRP 549919-20.

REFERENCES

[1] Gyarmathy, "Comprehensive guide to iot statistics you need to know in 2021," https://www.vxchnge.com/blog/iot-statistics, 2020.

[2] Bittencourt, Lopes *et al.*, "Towards virtual machine migration in fog computing," in *2015 10th International Conference on P2P, Parallel, Grid, Cloud and Internet Computing (3PGCIC)*, 2015, pp. 1–8.

[3] Rejiba, Masip-Bruin, and Marín-Tordera, "A survey on mobility-induced service migration in the fog, edge, and related computing paradigms," *ACM Comput. Surv.*, vol. 52, no. 5, Sep. 2019. [Online]. Available: https://doi.org/10.1145/3326540

[4] Bellavista, Zanni, and Solimando, "A migration-enhanced edge computing support for mobile devices in hostile environments," in *2017 13th International Wireless Communications and Mobile Computing Conference (IWCMC)*, 2017, pp. 957–962.

[5] Gonçalves, Velasquez *et al.*, "Proactive virtual machine migration in fog environments," in *2018 IEEE Symposium on Computers and Communications (ISCC)*, 2018, pp. 00 742–00 745.

[6] Araújo, Sousa *et al.*, "Cmfog: Proactive content migration using markov chain and madm in fog computing," in *2020 IEEE/ACM 13th International Conference on Utility and Cloud Computing (UCC)*, 2020, pp. 112–121.

[7] Puliafito, Gonçalves *et al.*, "Mobfogsim: Simulation of mobility and migration for fog computing," *Simulation Modelling Practice and Theory*, vol. 101, p. 102062, 2020, modeling and Simulation of Fog Computing. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1569190X19301935

[8] Khelifi, Luo *et al.*, "An optimized proactive caching scheme based on mobility prediction for vehicular networks," pp. 1–6, 2018.

[9] Tan, Wang *et al.*, "Delay-optimal task offloading for dynamic fog networks," in *ICC 2019 - 2019 IEEE International Conference on Communications (ICC)*, 2019, pp. 1–6.

[10] Abani, Braun, and Gerla, "Proactive caching with mobility prediction under uncertainty in information-centric networks," pp. 88–97, 09 2017.

[11] Zhao, Karimzadeh *et al.*, "Mobility management with transferable reinforcement learning trajectory prediction," *IEEE Transactions on Network and Service Management*, vol. 17, no. 4, pp. 2102–2116, 2020.

[12] Godinho, Silva *et al.*, "Energy and latency-aware resource reconfiguration in fog environments," in *2020 IEEE 19th International Symposium on Network Computing and Applications (NCA)*, 2020, pp. 1–8.

[13] Skarlat, Nardelli *et al.*, "Towards qos-aware fog service placement," in *2017 IEEE 1st International Conference on Fog and Edge Computing (ICFEC)*, 2017, pp. 89–96.

[14] Minh, Nguyen *et al.*, "Toward service placement on fog computing landscape," in *2017 4th NAFOSTED Conference on Information and Computer Science*, 2017, pp. 291–296.

[15] Sousa, Pentikousis, and Curado, "Methodical: Towards the next generation of multihomed applications," *Computer Networks*, vol. 65, 06 2014.

[16] Cattrysse and Van Wassenhove, "A survey of algorithms for the generalized assignment problem," *European Journal of Operational Research*, vol. 60, no. 3, pp. 260–272, 1992. [Online]. Available: https://www.sciencedirect.com/science/article/pii/037722179290077M

[17] Cplex, "V12. 1: User's manual for cplex," *International Business Machines Corporation*, vol. 46, no. 53, p. 157, 2009.

[18] Codecá, Frank *et al.*, "Luxembourg SUMO Traffic (LuST) Scenario: Traffic Demand Evaluation," *IEEE Intelligent Transportation Systems Magazine*, vol. 9, no. 2, pp. 52–63, 2017.

APPENDIX A

A. KKT Conditions for Constraints

$$\lambda_n^{(1)*}\left((\sum_{d=0}^{D} p_{dn}^*) - 1\right) = 0 \qquad \forall n \in N \tag{7a}$$

$$\lambda_d^{(2)*}\left((\sum_{n=0}^{N} p_{dn}^*) - 1)\right) = 0 \qquad \forall d \in D \tag{7b}$$

$$\mu_n^{(1)*}\left((\sum_{\substack{d=0 \\ x_{dn}\neq 0}}^{D} p_{dn}^* e_d) - E_n\right) = 0 \qquad \forall n \in N \tag{7c}$$

$$\mu_n^{(2)*}\left((\sum_{d=0}^{D} p_{dn}^* m_d) - M_n\right) = 0 \qquad \forall n \in N \tag{7d}$$

$$\theta_{dn}^*(p_{dn}x_{dn} - \tau) = 0 \quad x_{dn} \neq 0 \quad \forall d \in D \quad \forall n \in N \tag{7e}$$

$$\omega_{dn}^{(1)*}(p_{dn}^* - 1) = 0 \qquad \forall d \in D \quad \forall n \in N \tag{7f}$$

$$\omega_{dn}^{(2)*}(p_{dn}^*) = 0 \qquad \forall d \in D \quad \forall n \in N \tag{7g}$$

B. Derivations KKT Conditions for Lagrange

$$\frac{\partial}{\partial p_{dn}} = -x_{dn} + \lambda_n^{(1)*} + \lambda_d^{(2)*} + \mu_n^{(1)*}e_d + \mu_n^{(2)*}m_d + \theta_{dn}^* x_{dn} + \omega_{dn}^{(1)*} - \omega_{dn}^{(2)*} = 0 \tag{8}$$

$$-x_{dn}(1-\theta_{dn}^*) + \lambda_n^{(1)*} + \lambda_d^{(2)*} + \mu_n^{(1)*}e_d + \mu_n^{(2)*}m_d = \omega_{dn}^{(2)*} - \omega_{dn}^{(1)*} \tag{9a}$$

by multiplying p_{dn}^* in both sides

$$[-x_{dn}(1-\theta_{dn}^*) + \lambda_n^{(1)*} + \lambda_d^{(2)*} + \mu_n^{(1)*}e_d + \mu_n^{(2)*}m_d]p_{dn}^* = \omega_{dn}^{(2)*}p_{dn} - \omega_{dn}^{(1)*}p_{dn}^* \tag{9b}$$

using (7f) $-\omega_{dn}^{(1)*}p_{dn}^* = -\omega_{dn}^{(1)*}$ and using (7g) $\omega_{dn}^{(2)}p_{dn}^* = 0$

$$[-x_{dn}(1-\theta_{dn}^*) + \lambda_n^{(1)*} + \lambda_d^{(2)*} + \mu_n^{(1)*}e_d + \mu_n^{(2)*}m_d]p_{dn}^* = -\omega_{dn}^{(1)*} \tag{9c}$$

inserting into (7f)

$$[-x_{dn}(1-\theta_{dn}^*) + \lambda_n^{(1)*} + \lambda_d^{(2)*} + \mu_n^{(1)*}e_d + \mu_n^{(2)*}m_d] p_{dn}^*(p_{dn}^* - 1) = 0 \tag{9d}$$

C. Solution Analysis

1) Expected diff delay smaller than the rest :

$$x_{dn}(1-\theta_{dn}^*) < \lambda_n^{(1)*} + \lambda_d^{(2)*} + \mu_n^{(1)*}e_d + \mu_n^{(2)*}m_d \tag{10a}$$

as $\omega_{dn}^{(1)*} \geq 0$, then $\omega_{dn}^{(1)*} = 0$, which will result in $p_{dn}^* = 0$

2) Expected diff delay bigger than the rest :

$$x_{dn}(1-\theta_{dn}^*) > \lambda_n^{(1)*} + \lambda_d^{(2)*} + \mu_n^{(1)*}e_d + \mu_n^{(2)*}m_d \tag{11a}$$

as $\omega_{dn}^{(1)*} \geq 0$, then $\omega_{dn}^{(1)*} > 0$, which will result in $p_{dn}^* = 1$

3) Expected diff delay equal to the rest :

if

$$x_{dn}(1-\theta_{dn}^*) = \lambda_n^{(1)*} + \lambda_d^{(2)*} + \mu_n^{(1)*}e_d + \mu_n^{(2)*}m_d \tag{12a}$$

then

$$0 < p_{dn} < 1 \tag{12b}$$

from (7e)

$$\theta_{dn}^*(p_{dn}x_{dn} - \tau) = 0 \tag{12c}$$

$$\theta_{dn}^* \neq 0 \tag{12d}$$

$$p_{dn}x_{dn} = \tau \tag{12e}$$

$$p_{dn} = \tau / x_{dn} \tag{12f}$$